

Optimizing SGEMM: A High-Performance Approach to Matrix Multiplication

Aaron Ang Jerry Ma
University of California, San Diego

Abstract

We present a high-performance SGEMM (single-precision general matrix multiply) CUDA kernel that combines global memory coalescing, shared-memory tiling, and two-dimensional block tiling. Using 16×16 thread blocks that each compute an 8×8 output tile—a 128×128 tile per block—with an inner K -tile of 32, the kernel exposes instruction-level parallelism through register-blocked outer products and `#pragma unroll`. On an NVIDIA T4 it reaches 4779.9 GFlops at $N = 2048$, approaching cuBLAS, and we analyze how occupancy, register pressure, and arithmetic intensity shape performance—including the degradation that occurs when N is not a multiple of the tile dimension.

1 Development Flow

Our SGEMM implementation achieves efficiency through three key optimization strategies: global memory coalescing, shared memory utilization, and two-dimensional block tiling. Our final implementation is as follows. We employ a thread block dimension of 16×16 (`BLOCKDIM = 16`), resulting in 256 threads per block. Each thread is responsible for computing an 8×8 block (`TILESCALE_M = TILESCALE_N = 8`), which means each thread block processes a 128×128 submatrix of C (`TILEDIM_M = TILEDIM_N = 128`). The tile dimension is derived from `BLOCKDIM  $\times$  TILESCALE = 16  $\times$  8`. We split every `TILEDIM_M` rows of A and `TILEDIM_N` columns of B into sub-block tiles and iterate over them, as shown in Figure 1.

Each sub-block tile has a dimension (`TILEDIM_K`) of 32. We chose this value to remain within our shared memory threshold of 64 KB while enabling efficient memory coalescing patterns. In each iteration over a sub-block tile, all threads in the thread block collaboratively load values from global to shared memory before computing the outer product for the `TILEDIM_M  $\times$  TILEDIM_N` submatrix, as shown in Figure 1.

As shown in Figure 2, each thread performs `TILEDIM_K = 32` matrix outer products. For each iteration over K , individual

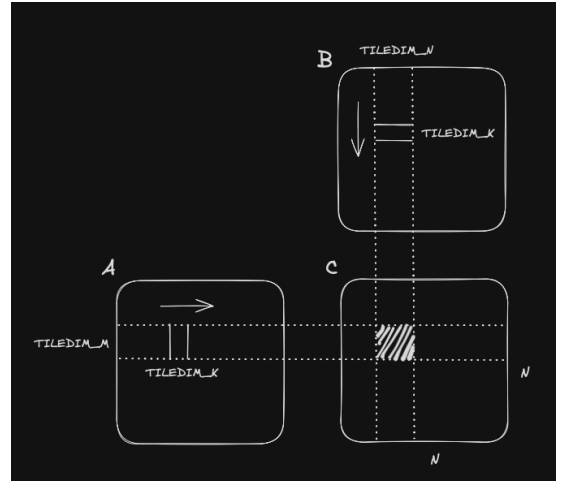


Figure 1: Iterate over sub-block tiles to accumulate the outer product of submatrix C .

threads load 8 values from block tile A and 8 from block tile B into registers. They then execute nested loops to accumulate the 8×8 result locally before writing the final results back to matrix C in global memory. Computing the outer product exposes substantial instruction-level parallelism (ILP) since there are no dependencies between instructions here. Paired with the `#pragma unroll` directive, we can guide the compiler to perform loop unrolling to achieve this ILP while keeping the code neat and flexible across various configurations.

To handle edge cases where matrix dimensions aren't multiples of 128, we implement bounds checking. During shared memory loading, we calculate global row and column positions as the product of thread block coordinates and `TILEDIM`, validating these bounds before transferring data from global to shared memory. This prevents result corruption from out-of-bounds data. Similarly, when writing results, we check bounds to ensure memory safety at matrix edges. The code is shown in Figures 3 and 4.

Our development progressed through several stages. We

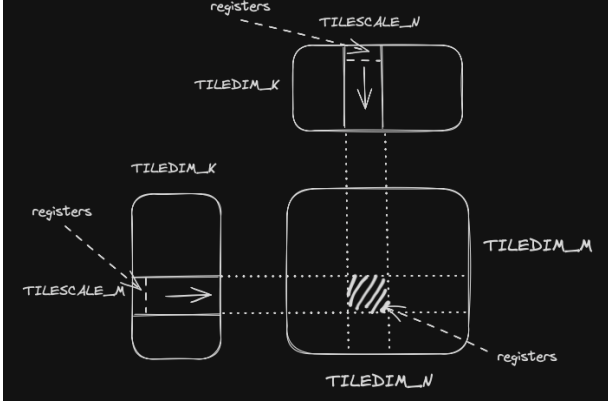


Figure 2: Each thread calculates a $\text{TILESCALE_M} \times \text{TILESCALE_N}$ outer product.

```
// populate the SMEM caches
#pragma unroll
for (uint loadOffset = 0; loadOffset < TILEDIM_M; loadOffset +=
    strideA) {
    const uint globalRowA = globalRowOffset + innerRowA +
        loadOffset;
    const uint globalColA = bkIdx + innerColA;
    As[innerRowA + loadOffset][innerColA] =
        (globalRowA < N && globalColA < N)
        ? A[(innerRowA + loadOffset) * N + innerColA]
        : 0.0f;
}
#pragma unroll
for (uint loadOffset = 0; loadOffset < TILEDIM_K; loadOffset +=
    strideB) {
    const uint globalRowB = bkIdx + innerRowB + loadOffset;
    const uint globalColB = globalColOffset + innerColB;
    Bs[innerRowB + loadOffset][innerColB] =
        (globalRowB < N && globalColB < N)
        ? B[(innerRowB + loadOffset) * N + innerColB]
        : 0.0f;
}
__syncthreads();
```

Figure 3: Bounds checking when populating shared memory.

started with shared memory optimization, achieving a 50% improvement over the naive kernel. We then implemented 1-D block tiling where each thread calculated a column of results, nearly doubling performance. The transition to two-dimensional block tiling was the hardest as we misunderstood the definitions of blocks and tiles. We tried various configurations but performance dropped significantly below that of 1-D block tiling. We believe it is due to bad coalesced loads when loading to shared memory and performing too many computations per thread. Thankfully, we clarified with TA Thomas during office hours that a block tile should represent the values computed by an entire thread block. With this insight and guidance from CUTLASS [3] and Simon Boehm’s blog [5], we successfully implemented our final solution.

2 Results

Throughout this section, we use the shorthand T_M , T_N , and T_K for TILEDIM_M , TILEDIM_N , and TILEDIM_K ; B_x and B_y for

```
// write out the results
#pragma unroll
for (uint resIdxM = 0; resIdxM < TILESCALE_M; ++resIdxM) {
    #pragma unroll
    for (uint resIdxN = 0; resIdxN < TILESCALE_N; ++resIdxN) {
        const uint tileRow = threadRow * TILESCALE_M + resIdxM;
        const uint tileCol = threadCol * TILESCALE_N + resIdxN;
        if (globalRowOffset + tileRow < N &&
            globalColOffset + tileCol < N) {
            C[tileRow * N + tileCol] =
                threadResults[resIdxM][resIdxN];
        }
    }
}
```

Figure 4: Bounds checking when writing results back to C .

the block dimensions BLOCKDIM_X and BLOCKDIM_Y ; and S_N for the tile scale TILESCALE_N .

(a) Configuration and access-pattern constraints. We fix $T_K = 32$, which means for each block tile, we divide N into N/T_K subtiles of A of size $T_M \times T_K$, and in each iteration, we perform the outer product $A_{T_M:N} \otimes B_{N:T_N}$.

The thread block and tile sizes are limited by the access pattern of A when loading into shared memory:

$$B_x \times B_y \geq T_K.$$

Similarly for B :

$$B_x \times B_y \geq T_N = S_N \times B_y.$$

The above equations can be simplified to:

$$B_x \geq S_N.$$

(b) Occupancy analysis. Based on Figure 5, the optimal thread block size is 16×16 across all N . However, we observed some irregular performance behaviors when $N = 256$. We analyze the potential reasons for the performance differences from the perspective of the number of warps and the possible maximum occupancy, as summarized in Table 1.

One intuitive explanation for why the configurations $\text{bx}=32, \text{by}=32, \text{tm}=2, \text{tn}=2$ and $\text{bx}=16, \text{by}=16, \text{tm}=4, \text{tn}=4$ perform better at $N = 256$ is that they require more warps, which leads to more concurrent SM usage.

The second reason could be the limitation of register file size. For the configurations $\text{bx}=16, \text{by}=16, \text{tm}=8, \text{tn}=8$ and $\text{bx}=8, \text{by}=8, \text{tm}=8, \text{tn}=8$, each thread utilizes $8 + 8 + 8 \times 8 = 80$ registers. The NVIDIA T4 GPU supports a maximum of 256 KB register files (equivalent to 64K float registers) per SM and a maximum of 32 concurrent warps per SM.

For this implementation, the register file size becomes the limiting factor for occupancy when N is large because 32 warps would require:

$$32 \text{ warps} \times 32 \text{ threads} \times 80 \text{ regs} = 81,920 \text{ registers},$$

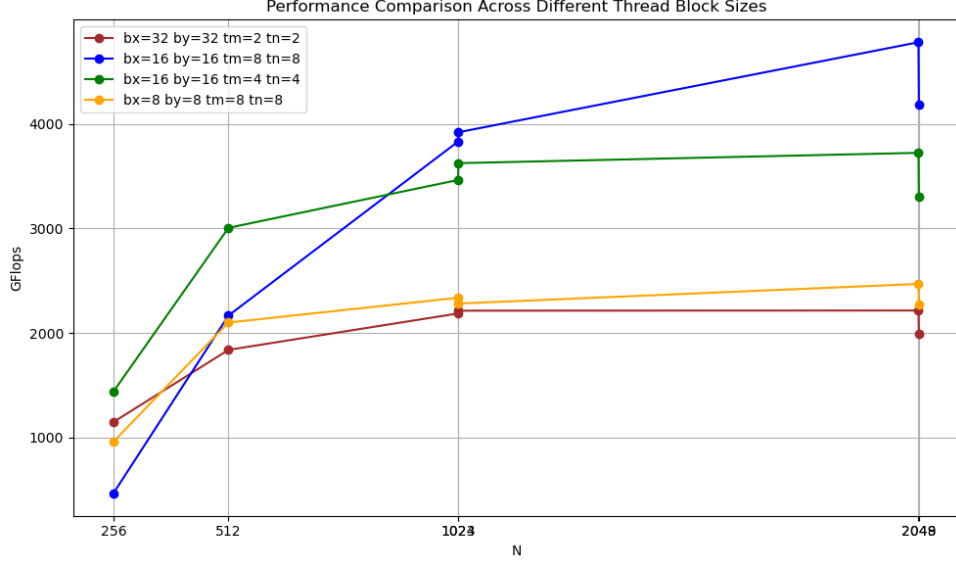


Figure 5: Performance across various block dimensions and tile scales.

Table 1: Occupancy analysis for several thread-block and tile configurations at $N = 256$ on the NVIDIA T4.

Configuration	Tile Dim.	Thread Blocks	Threads/Block	Threads	Regs/Thread	Warps	Max Occ.
bx=32 by=32 tm=2 tn=2	64	16	1024	16384	8	512	100%
bx=16 by=16 tm=8 tn=8	128	4	256	1024	80	32	80%
bx=8 by=8 tm=8 tn=8	64	16	64	1024	80	32	80%
bx=16 by=16 tm=4 tn=4	64	16	256	4096	24	128	100%

which exceeds the maximum of 64K float registers. As a result, the maximum number of active warps is given by:

$$\frac{\text{total registers}}{\text{registers per warp}} = \frac{64K}{80 \times 32} = 25.6 \text{ warps},$$

which leads to an occupancy of:

$$25.6/32 \approx 80\%.$$

Thus, the SM is underutilized at approximately 80% occupancy due to register limitations. The total shared memory size is $32 \times 16 \times 8 \times 2 \times 4B = 32KB$, which is within the 64 KB limitation.

For the configurations $bx=32, by=32, tm=2, tn=2$ and $bx=16, by=16, tm=4, tn=4$, the register file size is no longer a limitation to occupancy; the occupancy is 100%.

We noticed that high occupancy is not always needed to reach high performance when the arithmetic intensity q is small or large. q is given by:

$$q = \frac{f}{m},$$

where f is the number of floating point operations, which is $2N^3$, and m is the number of memory operations. Each

block tile loads $N(T_M + T_N)$ values from matrices A and B into shared memory A_S and B_S , and the number of block tiles is $N^2/(T_M T_N)$. We also perform N^2 store operations back to global memory. Thus, we can simplify q as:

$$q = \frac{2N^3}{\frac{2N^3(T_M + T_N)}{T_M T_N} + N^2} = \frac{2N T_M T_N}{N(T_M + T_N) + T_M T_N}.$$

If we consider the square cases where $T_M = T_N$, we can simplify q as:

$$q = \frac{2N T_M}{2N + T_M}.$$

When T_M is large, we become compute-bound and higher occupancy would not lead to better throughput, as explained in the cusp behavior described by Volkov [2]. For the configuration $bx=16, by=16, tm=8, tn=8$, it has the highest T_M of 128, yet it outperforms other configurations despite its lower occupancy.

(c) Peak performance by matrix size. Table 2 documents the peak GFlops achieved and the corresponding thread block size for each matrix size.

Table 2: Peak GFlops and the corresponding thread block size for each matrix size.

N	Peak GF	Thread Block Size
256	1148.9	bx=16 by=16 tm=4 tn=4
512	2166.9	bx=16 by=16 tm=8 tn=8
1023	3829.8	bx=16 by=16 tm=8 tn=8
1024	3919.6	bx=16 by=16 tm=8 tn=8
2048	4779.9	bx=16 by=16 tm=8 tn=8
2049	4181.0	bx=16 by=16 tm=8 tn=8

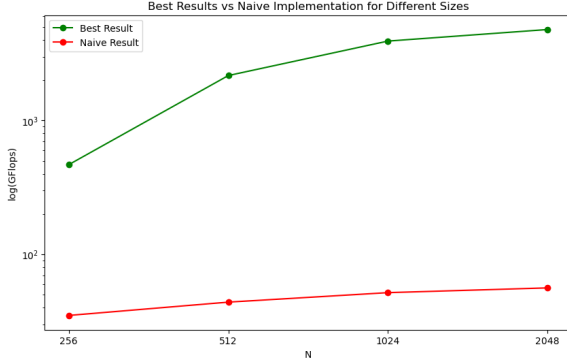


Figure 6: log(GFlops) comparison with the naive implementation.

3 Comparison to Naive

Figure 6 compares the throughput of our kernel against the naive implementation on a logarithmic scale, and Figure 7 shows the corresponding speedup.

4 Analysis

(a) Performance across implementations. Figure 8 shows performance across our different implementations.

(b) Resemblance to cuBLAS. We noticed that the performance curve of our 2D-tiling-64-outputs method closely resembles that of cuBLAS, with both showing fluctuations before converging to around 4000+ GFlops. One possible explanation for this similarity is that both methods are based on the block-tile approach.

(c) Performance anomalies. We noticed two unusual behaviors in performance.

1) Performance degradation when N is not a multiple of 128. Our 2D-tiling-64-outputs implementation and cuBLAS exhibit a significant performance drop when the matrix size N is not a multiple of 128. In contrast, other implementations, such as 1-D blocking and naive methods, show much less fluctuation. This disparity arises because our implementation is more complex, involving additional branching conditions that cause more overhead during control divergence.

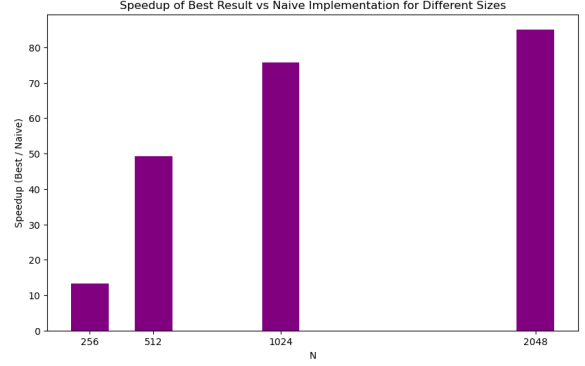


Figure 7: Speedup over the naive implementation.

During boundary checks, when we attempt to fill the A_s and B_s matrices with zeros for out-of-bounds accesses, some threads within a warp follow different execution paths. As a result, the GPU must re-execute the warp for threads that take the alternate path, causing performance degradation. For example, $N = 1024$ benefits from perfect tiling and memory alignment. However, $N = 1023$ leads to slight tiling imbalances, resulting in minor inefficiencies, while $N = 1025$ suffers from both uneven tiling and memory misalignment, significantly reducing warp utilization. This results in a performance drop from over 3900 GFlops to around 2900 GFlops.

A similar performance drop occurs when $N = 1199$, 1200 , and 1201 . However, all these matrix sizes fail to reach good memory alignment, and there is minimal difference in performance across these three values. The varying degrees of performance degradation suggest using different tile sizes, with each matrix size resulting in a different level of tiling inefficiency.

2) Performance degradation when N is greater than 768. When $N > 768$, we observe that performance no longer improves significantly and, in some cases, even degrades for certain matrix sizes. One possible reason is that the kernel is increasingly memory-bound as N increases. Since we fixed our kernel to compute an 8×8 block per thread, computational intensity (q) plateaus after $N = 1000$. We use the same formula for q as in Section 2, giving us a sawtooth logarithmic graph shown in Figure 9:

$$q = \frac{2N^3}{\lceil N/T_M \rceil^2 \cdot 2NT_M + N^2}.$$

The shape of the graph confirms our hypothesis that the performance of our matrix-multiply implementation does not scale linearly with N and dips when N is not a multiple of $TILEDIM_M$.

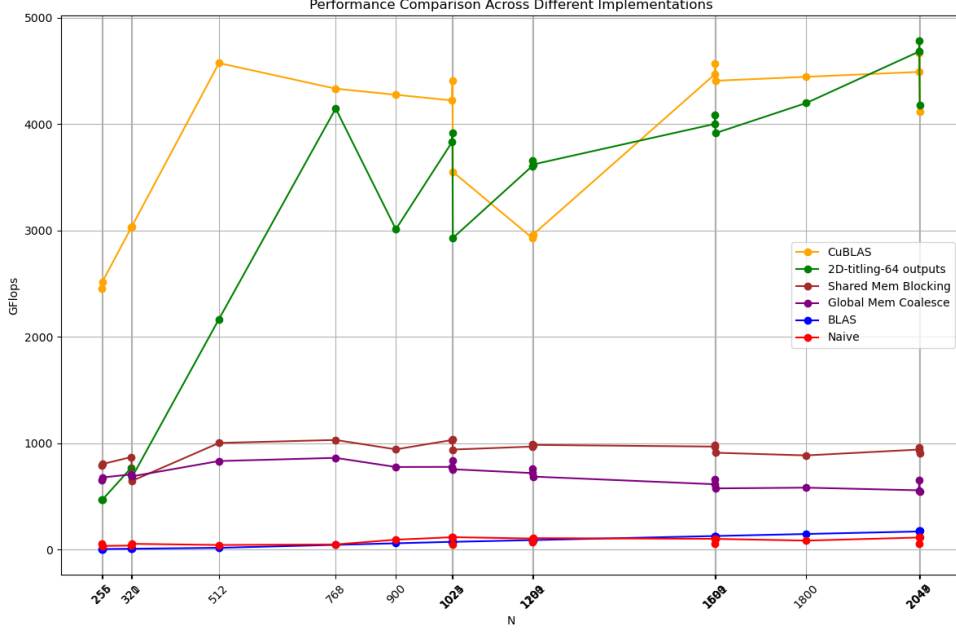


Figure 8: Performance across different implementations.

5 Calculations

(a) Peak performance and arithmetic intensity. Peak performance is the total operations per second that a GPU can perform, assuming all its cores execute their maximum number of ops per cycle, where frequency is cycles per second per core. Thus,

$$\begin{aligned}
 \text{peak} &= \text{SMs} \times \frac{\text{cores}}{\text{SM}} \times \text{freq} \times \frac{\text{ops}}{\text{cycle}} \\
 &= 40 \times 64 \times (1.5 \times 10^9) \times 2 \\
 &= 7680 \text{ GFlops/s.}
 \end{aligned}$$

For $n = 2048$, we achieved 4779.9 GFlops. To calculate the arithmetic intensity q , we divide the number of flops ($2n^3$) by the number of memory operations ($2n^3/\text{TILEDIM_M}$ loads and n^2 stores), as shown in Figure 10.

(b) Effect of reduced memory bandwidth. $q = \text{peak flops/bandwidth} = 7680/320 = 24.0$ Flops/byte. The new $q = 7680/220 = 34.9$ Flops/byte. The value of q increased. This means we now need greater arithmetic intensity to achieve peak performance, since we are limited by the speed of memory operations (Figure 11).

6 Potential Future Work

Given more time, we would have explored interleaved computation as a potential optimization. Since our current implementation already achieved strong performance, we decided not to pursue it. This technique would have required a substantial

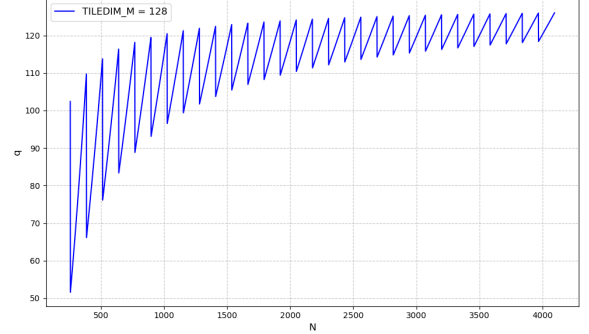


Figure 9: Computational intensity of our kernel over N .

departure from our outer-product approach, which focuses on computing contiguous 2-D blocks. The interleaved method would mean redesigning our thread block dimensions and tile sizes, effectively requiring an architectural rethink rather than an incremental improvement to our existing design.

7 Extra Credit

We created a non-square matrix multiplication kernel with the following header in `mmpy_kernel.cu`:

```

__global__ void matMulNonSquare(
    int M, int N, int K,
    _FTYPE_ *__restrict__ C,
    _FTYPE_ *__restrict__ A,
    _FTYPE_ *__restrict__ B)

```

This required three changes to our original implementation.

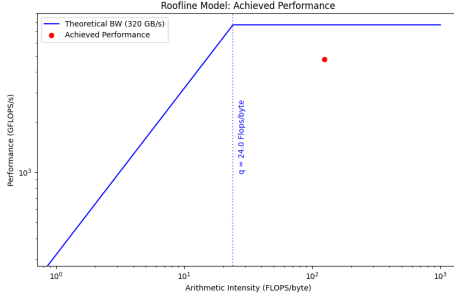


Figure 10: Roofline model: achieved performance at $N = 2048$ ($q = 24.0$ Flops/byte).

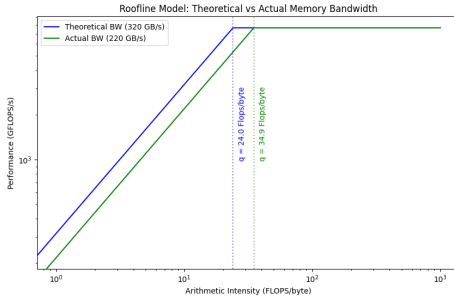


Figure 11: Roofline model: theoretical versus actual memory bandwidth ($q = 24.0$ versus 34.9 Flops/byte).

First, we updated the outer loop bound from N to K , since K represents the shared dimension between matrices A and B used to iterate the sub-block tiles (refer to Figure 1). Second, we modified the shared memory loading bounds to check against M for matrix A and N for matrix B . Finally, when writing results back to matrix C , we updated our bounds checking to compare the global row offset against M and the global column offset against N , since the output is now an $M \times N$ matrix.

To optimize our non-square matrix multiplication kernel, we could implement several changes. Instead of using fixed 128×128 tile sizes, we could dynamically adjust tile dimensions M , N , and K . For example, if M is much larger than N , we can scale `TILEDIM_M` by a larger factor (using `TILESCALE_M`) up to the shared memory limit. Essentially, our outer product dimensions would be proportional to M and N , maximizing the amount of work done per thread. When K is much larger than M or N , we could increase `TILEDIM_K` (while adjusting other parameters) to maximize shared memory utilization. These changes would enable the matrix multiplication computation to remain highly performant across varying input sizes.

References

- [1] V. Volkov. *Understanding Latency Hiding on GPUs*. Ph.D. thesis, University of California, Berkeley, Technical Report UCB/EECS-2016-143, 2016. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-143.html>.
- [2] V. Volkov. Better performance at lower occupancy. In *GPU Technology Conference (GTC)*, 2010. https://www.nvidia.com/content/gtc-2010/pdfs/2238_gtc2010.pdf.
- [3] NVIDIA. CUTLASS: Fast linear algebra in CUDA C++. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/>.
- [4] Must know technique in GPU computing: Tiled matrix multiplication in CUDA C. <https://www.youtube.com/watch?v=Q3GgbfGTnVc>.
- [5] S. Boehm. How to optimize a CUDA matmul kernel for cuBLAS-like performance: A worklog. <https://siboehm.com/articles/22/CUDA-MMM>.